# Regular Expression: Text Normalization

# Introduction

- Regular Expressions (regex) for Text Normalization, is a key step in preprocessing text for NLP tasks like sentiment analysis.
- Text normalization is the process of transforming text into a consistent, standard format. Common tasks include:Lowercasing, Removing punctuation, Removing special characters, Removing numbers, Removing extra whitespace etc.
- Normalization methods: stemming, lemmatization, or contraction expansion

# Common Regex Patterns for Text Normalization

| Task | Regex Pattern | Description |
|---|---|---|
| Lowercasing | re.sub(r'[A-Z]' | Converts all uppercase to lowercase |
| Remove punctuation | [^\w\s] | Removes anything not word or space |
| Remove numbers | \d+ | Removes all digits |
| Remove special chars | [^a-zA-Z0-9\s] | Keeps only letters, numbers, space |
| Remove extra whitespace | \s+ | Replaces multiple spaces with one |

# Contd...

| Task | Regex Pattern | Description |
|---|---|---|
| Remove URLs | `https?://\S+ | Matches http://, https://, www. |
| Remove HTML Tags | <[^>]+> | Removes <tags> like <p>,  |
| Remove Emojis | [^\x00-\x7F]+ | Removes non-ASCII (emojis, symbols) |
| Replace Contractions | won\'t → will not | Expands can't → cannot |

# Naïve bayes Algorithm for Sentiment classification

# Introduction

Naive Bayes applies Bayes' Theorem with the "naive" assumption that all features (words in text) are conditionally independent given the class (sentiment).

$$\text{Bayes Theorem: } P(y|X) = P(X|y) * P(y) / P(X)$$

Where:

y = sentiment class (positive/negative)

X = text features (words)

- Marginal Probability **always remains the same** in the calculation of Naive Bayes classifier.
- Ignore the calculation of **marginal probability**.
- In Naive Bayes Classifier, determine the class of data points based on the value of the **greatest posterior probability.**

# Implementation Steps

- **Data Preprocessing:**
  - Tokenization (split text into words)
  - Remove stop words, punctuation
  - Stemming/Lemmatization
  - Create word frequency counts
- **Feature Extraction:**
  - Create a vocabulary of all unique words
  - Represent each document as a vector of word counts (Bag of Words)

# Contd...

- **Training Phase:**
  - Calculate prior probabilities P(y) for each sentiment class
  - Calculate likelihoods P(word | y) for each word in each class
- **Classification Phase:**
  - For a new document, calculate posterior probability for each class
  - Assign the class with highest probability

# Types of Naive Bayes

- **Multinomial Naive Bayes:**
  - Most common for text classification
  - Uses word frequency counts
- **Bernoulli Naive Bayes:**
  - Uses binary word presence/absence features
  - Often performs better on short texts
- **Gaussian Naive Bayes:**
  - For continuous features (not typically used in text)

# Example 1

Consider the following dataset:

| Document | Sentiment |
|---|---|
| "I love this movie" | Positive |
| "This movie is great" | Positive |
| "I hate this movie" | Negative |
| "This movie is terrible" | Negative |

**Step 2: Preprocessing**

**Tokens:**

- Positive: "I", "love", "this", "movie", "is", "great"
- Negative: "I", "hate", "this", "movie", "is", "terrible"
- Vocabulary: "I", "love", "this", "movie", "is", "great", "hate", "terrible"

## Step 3: Calculating Probabilities

1. Prior Probabilities
   - P(Positive) = 2/4 = 0.5
   - P(Negative) = 2/4 = 0.5
2. Likelihood with Laplace Smoothing
   - Vocabulary Size (V) = 8 (all unique words)
   - Total words in Positive = 6
   - Total words in Negative = 6

$$P(word|class) = \frac{count(word, class) + 1}{\text{Total words in class} + V}$$

**Step 4: Classifying a New Document**

New Document: "I love this movie"

We want to find:

$$P(\text{Positive}|\text{"I love this movie"}) \propto P(\text{Positive}) \times P(\text{I}|\text{Positive}) \times P(\text{love}|\text{Positive}) \times P(\text{this}|\text{Positive}) \times P(\text{movie}|\text{Positive})$$

**Positive Probability Calculation:**

P(Positive) × P(I|Positive) × P(love|Positive) × P(this|Positive) × P(movie|Positive)

$$= 0.5 \times \frac{2}{14} \times \frac{2}{14} \times \frac{2}{14} \times \frac{2}{14}$$

$$= 0.5 \times \left(\frac{2}{14}\right)^4$$

$$= 0.5 \times \frac{16}{38416}$$

$$\approx 0.5 \times 0.000416 = 0.000208$$

**Negative Probability Calculation:**

P(Negative) × P(I|Negative) × P(love|Negative) × P(this|Negative) × P(movie|Negative)

$$= 0.5 \times \frac{2}{14} \times \frac{1}{14} \times \frac{2}{14} \times \frac{2}{14}$$

$$= 0.5 \times \frac{8}{38416}$$

$$\approx 0.5 \times 0.000208 = 0.000104$$

**Step 5: Final Decision**

- P(Positive) ≈ 0.000208
- P(Negative) ≈ 0.000104

Since P(Positive) > P(Negative),

The document "I love this movie" is classified as Positive.

# Example 2

- Naive Bayes is a popular classification algorithm based on Bayes' theorem, especially useful for tasks like sentiment analysis.
- Consider a simple example where we classify short text as either Positive or Negative sentiment.

# Sample Data Set

| Review Text | Contains "great" | Contains "poor" | Sentiment |
|---|---|---|---|
| Great movie | 1 | 0 | Positive |
| Great plot | 1 | 0 | Positive |
| Poor acting | 0 | 1 | Negative |
| Terrible movie | 0 | 0 | Negative |
| Not great | 1 | 0 | Negative |
| Poor direction | 0 | 1 | Negative |

# Contd...

- **Step 1: Calculate Class Probabilities (P(Y))**
  - Total reviews = 6
  - Positive reviews = 2
  - Negative reviews = 4

$$P(\text{Positive}) = \frac{\text{Count(Positive)}}{\text{Total Reviews}} = \frac{2}{6} = 0.333$$

$$P(\text{Negative}) = \frac{\text{Count(Negative)}}{\text{Total Reviews}} = \frac{4}{6} = 0.666$$

**Step 2: Calculate Feature Probabilities (P(X | Y))**

- We compute the probability of each feature given the sentiment class.
- Since we have binary features (1 if word appears, 0 otherwise), we use Bernoulli Naive Bayes
- For Positive class (Y=Positive):
  - Total Positive reviews = 2
  - "great" appears in 2 Positive reviews
  - "poor" appears in 0 Positive reviews

# Contd...

**Using Laplace Smoothing (Add-1 Smoothing) to avoid zero probabilities:**

$$P(\text{"great"} = 1|\text{Positive}) = \frac{\text{Count}(\text{"great" in Positive}) + 1}{\text{Positive Reviews} + 2} = \frac{2+1}{2+2} = \frac{3}{4} = 0.75$$

$$P(\text{"poor"} = 1|\text{Positive}) = \frac{0+1}{2+2} = \frac{1}{4} = 0.25$$

# Contd...

- For Negative class (Y=Negative):
  - Total Negative reviews = 4
  - "great" appears in 1 Negative review
  - "poor" appears in 2 Negative reviews

$$P(\text{"great"} = 1|\text{Negative}) = \frac{1+1}{4+2} = \frac{2}{6} = 0.333$$

$$P(\text{"poor"} = 1|\text{Negative}) = \frac{2+1}{4+2} = \frac{3}{6} = 0.5$$

**Step 4: Classify a New Review**

Example: Classify the review "Great poor acting"

Features:

- "great" = 1
- "poor" = 1

# Contd...

**Step 1: Compute Posterior Probabilities (Using Bayes' Theorem)**

Naive Bayes assumes feature independence, so:

$$P(Y|X) \propto P(Y) \times \prod_{i=1}^{n} P(X_i|Y)$$

$P(\text{Positive}|\text{"great"}=1,\text{"poor"}=1) \propto$
$P(\text{Positive}) \times P(\text{"great"}=1|\text{Positive}) \times P(\text{"poor"}=1|\text{Positive})$

$$= 0.333 \times 0.75 \times 0.25 = \mathbf{0.0625}$$

# Contd...

**For Negative Class:**

$$P(\text{Negative} \mid \text{"great"}=1, \text{"poor"}=1) \propto$$
$$P(\text{Negative}) \times P(\text{"great"}=1 \mid \text{Negative}) \times P(\text{"poor"}=1 \mid \text{Negative})$$

$$= 0.666 \times 0.333 \times 0.5 = \mathbf{0.111}$$

## Step 2: Normalize to Get Final Probabilities

$$\text{Total} = 0.0625 + 0.111 = 0.1735$$

$$P(\text{Positive}) = \frac{0.0625}{0.1735} \approx 0.36 \quad (36\%)$$

$$P(\text{Negative}) = \frac{0.111}{0.1735} \approx 0.64 \quad (64\%)$$

## Final Prediction:

The review "Great poor acting" is classified as Negative (64% confidence).

# Advantages for Sentiment Analysis

- Simple and fast to train
- Works well with high-dimensional text data
- Performs surprisingly well even with the naive independence assumption
- Handles irrelevant features relatively well

# Decision Tree Algorithm

# Introduction

- A **decision tree** is a supervised machine learning algorithm used for both classification and regression tasks. It creates a model that predicts the value of a target variable by learning simple decision rules inferred from the data features.

Key Components of a Decision Tree

- Root Node: The topmost decision node
- Internal Nodes: Decision nodes that split the data
- Leaf Nodes: Terminal nodes that make final predictions
- Branches: Outcomes of a decision node

## Decision Tree Algorithm Steps

- Select the best attribute using Attribute Selection Measures (ASM)
- Make that attribute a decision node and split the dataset
- Repeat the process for child nodes until:
  - ✓All tuples belong to the same class
  - ✓No remaining attributes
  - ✓No more instances left

## Attribute Selection Measures

- Information Gain: Based on entropy reduction
- Gini Index: Measures impurity
- Gain Ratio: Modification of information gain

# Contd...

Consider a simple dataset about weather conditions and whether to play Circket:

| Outlook | Temperature | Humidity | Windy | Cricket |
|---|---|---|---|---|
| Sunny | Hot | High | Less | No |
| Sunny | Hot | High | Heavy | No |
| Overcast | Hot | High | Less | Yes |
| Rainy | Mild | High | Less | Yes |
| Rainy | Cool | Normal | Less | Yes |
| Rainy | Cool | Normal | Heavy | No |
| Overcast | Cool | Normal | Heavy | Yes |

# Contd...

| Outlook | Temperature | Humidity | Windy | Cricket |
|---|---|---|---|---|
| Sunny | Mild | High | Less | No |
| Sunny | Cool | Normal | Less | Yes |
| Rainy | Mild | Normal | Less | Yes |
| Sunny | Mild | Normal | Heavy | Yes |
| Overcast | Mild | High | Heavy | Yes |
| Overcast | Hot | Normal | Less | Yes |
| Rainy | Mild | High | Heavy | No |

Step 1: Calculate Entropy of the Parent Node

- Total instances: 14
- Yes: 9, No: 5

Entropy = - (p_yes * log2(p_yes) + p_no * log2(p_no))

= - (9/14 * log2(9/14) + 5/14 * log2(5/14))

= - (0.642 * -0.637 + 0.357 * -1.485)

= - (-0.409 + -0.530)

= 0.940

## Step 2: Calculate Information Gain for Each Attribute

*1. Outlook Attribute:*

Sunny: 5 instances (Yes: 2, No: 3)

Overcast: 4 instances (Yes: 4, No: 0)

Rainy: 5 instances (Yes: 3, No: 2)

Entropy(Sunny) = - (2/5log2(2/5) + 3/5log2(3/5)) = 0.971

Entropy(Overcast) = 0 (pure node)

Entropy(Rainy) = - (3/5log2(3/5) + 2/5log2(2/5)) = 0.971

**Information Gain** = Entropy(parent) - **Σ** (weighted entropy of children)

= 0.940 - [(5/14)*0.971 + (4/14)*0 + (5/14)*0.971]

= 0.940 - [0.347 + 0 + 0.347]

= 0.246

## 2. *Temperature Attribute:*

- Hot: 4 instances (Yes: 2, No: 2)
- Mild: 6 instances (Yes: 4, No: 2)
- Cool: 4 instances (Yes: 3, No: 1)

- Entropy(Hot) = 1.0
- Entropy(Mild) = 0.918
- Entropy(Cool) = 0.811

**Information Gain** = 0.940 - [(4/14)*1 + (6/14)*0.918 + (4/14)*0.811]

= 0.940 - [0.286 + 0.394 + 0.232]

= 0.028

*3. **Humidity Attribute:***

- High: 7 instances (Yes: 3, No: 4)
- Normal: 7 instances (Yes: 6, No: 1)

Entropy(High) = 0.985

Entropy(Normal) = 0.592

**Information Gain** = 0.940 - [(7/14)*0.985 + (7/14)*0.592]

= 0.940 - [0.493 + 0.296]

= 0.151

## *4. Windy Attribute:*

- Heavy: 6 instances (Yes: 3, No: 3)
- Less: 8 instances (Yes: 6, No: 2)

Entropy(Heavy) = 1.0

Entropy(less) = 0.811

**Information Gain** = 0.940 - [(6/14)*1 + (8/14)*0.811]

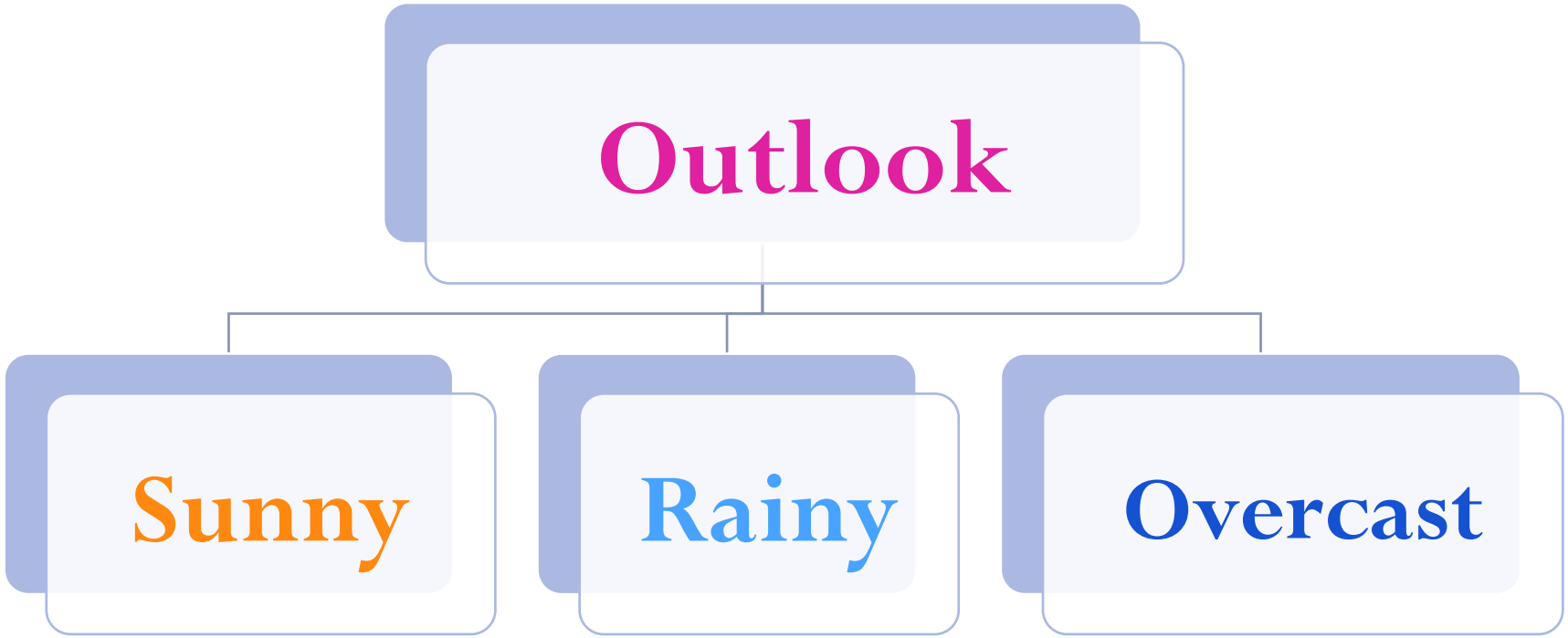
= 0.940 - [0.429 + 0.463]

= 0.048

## Step 3: Select Attribute with Highest Information Gain

- Outlook has the highest information gain (0.246), so we select it as our root node.

# Decision Tree - Root node



## Step 4: Build Sub-trees

- For the Overcast branch (pure node - all "Yes"), we stop.
- For Sunny branches, we repeat the process with remaining attributes.

| Temperature | Humidity | Windy | Cricket |
|---|---|---|---|
| Hot | High | Less | No |
| Hot | High | Heavy | No |
| Mild | High | Less | No |
| Cool | Normal | Less | Yes |
| Mild | Normal | Heavy | Yes |

Calculate Entropy of the Sunny Node

- Total instances: 5
- Yes: 2, No: 3

Entropy = - (p_yes * log2(p_yes) + p_no * log2(p_no))

= - (2/5 * log2(2/5) + 3/5 * log2(3/5))

= - (-0.528 -0.442)

= 0.97

***a. Temperature Attribute:***

- Hot: 2 instances (Yes: 0, No: 2)
- Mild: 2 instances (Yes: 1, No: 1)
- Cool: 1 instances (Yes: 1, No: 0)

- Entropy(Hot) = 0
- Entropy(Mild) = 1
- Entropy(Cool) = 0

**Information Gain** = 0. 97 - [(2/5)*0 + (2/5)*1 + (1/5)*0]

= 0.97 - [0+ 0.4 + 0]

= 0.57

***b. Humidity Attribute:***

- High: 3 instances (Yes: 0, No: 3)
- Normal: 2 instances (Yes: 2, No: 0)

Entropy(High) = 0

Entropy(Normal) = 0

**Information Gain** = 0.97 - [(3/5)*0 + (2/5)*0]

= 0.97 - [0]

= 0.97

***c. Windy Attribute:***

- Heavy: 2 instances (Yes: 1, No: 1)
- Less: 3 instances (Yes: 1, No: 2)

Entropy(Heavy) = 1

Entropy(less) = 0.917

**Information Gain** = 0.97 - [(2/5)*1 + (3/5)*0.917]

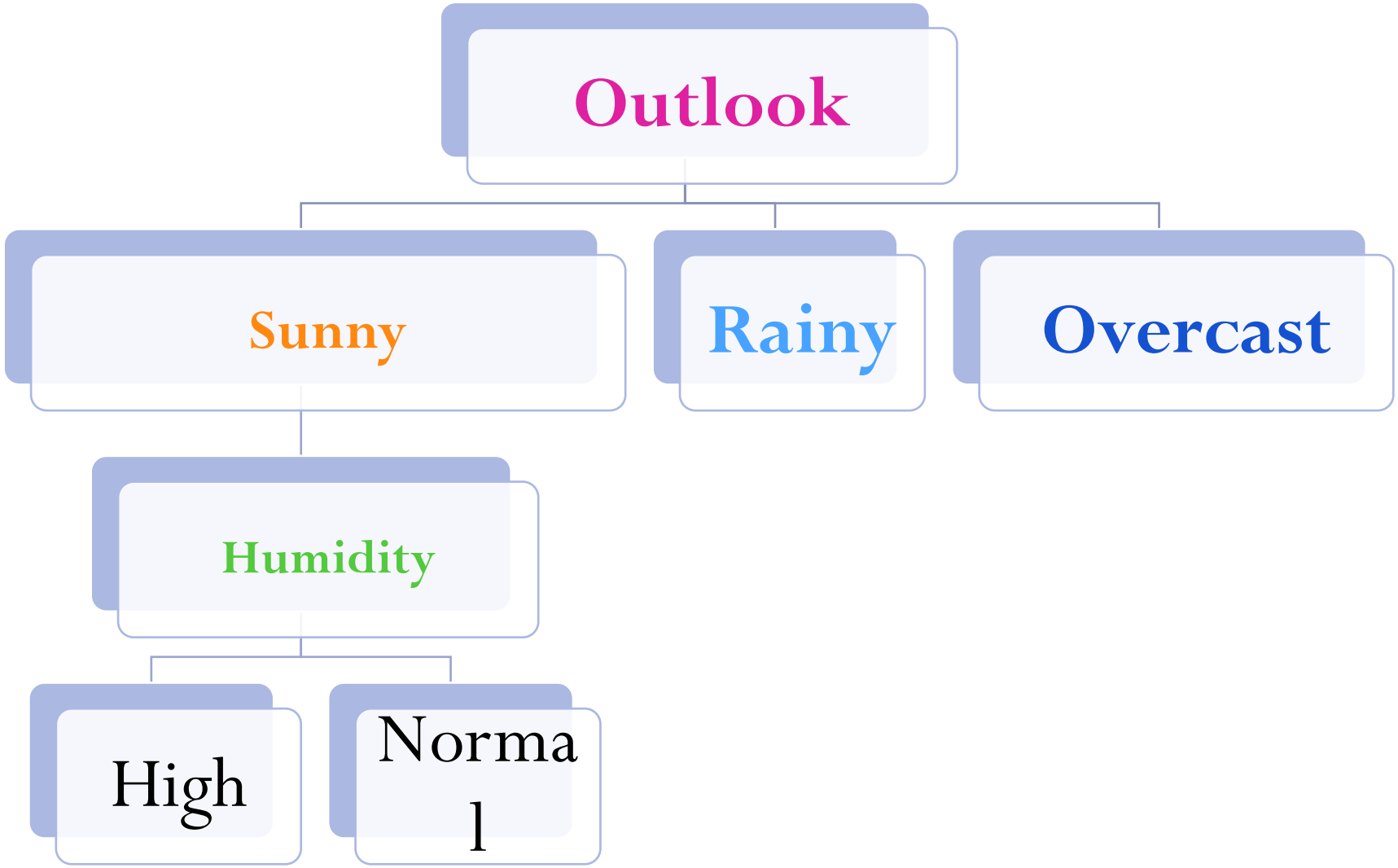
= 0.97 - [0.4+ 0.550]

= 0.02

**Sunny Branch:**

- Temperature, Humidity, Windy
- Calculate information gain for each: Humidity has highest Information gain

# Decision Tree



## Step 4: Build Sub-trees

- For Rainy branches, we repeat the process with remaining attributes.

| Temperature | Windy | Cricket |
|---|---|---|
| Mild | Less | Yes |
| Cool | Less | Yes |
| Cool | Heavy | No |
| Mild | Less | Yes |
| Mild | Heavy | No |

Calculate Entropy of the Rainy Node

- Total instances: 5
- Yes: 3, No:2

Entropy = - (p_yes * log2(p_yes) + p_no * log2(p_no))

= - (3/5 * log2(3/5) + 2/5 * log2(2/5))

= 0.97

***a. Temperature Attribute:***

- Mild: 3 instances (Yes: 2, No: 1)
- Cool: 2 instances (Yes: 1, No: 1)

- Entropy(Mild) = 0.917
- Entropy(Cool) = 1

**Information Gain** = 0. 97 - [(3/5)*0.917+ (2/5)*0.1]

= 0.97 - [ 0.6*0.917+0.4*1]

= 0.97 - 0.95

= 0.02

***b. Windy Attribute:***

- Heavy: 2 instances (Yes: 0, No: 2)
- Less: 3 instances (Yes: 3, No: 0)

Entropy(Heavy) = 0

Entropy(less) = 0

**Information Gain** = 0.97 - [(2/5)*0 + (3/5)*0]

= 0.97 - [0]

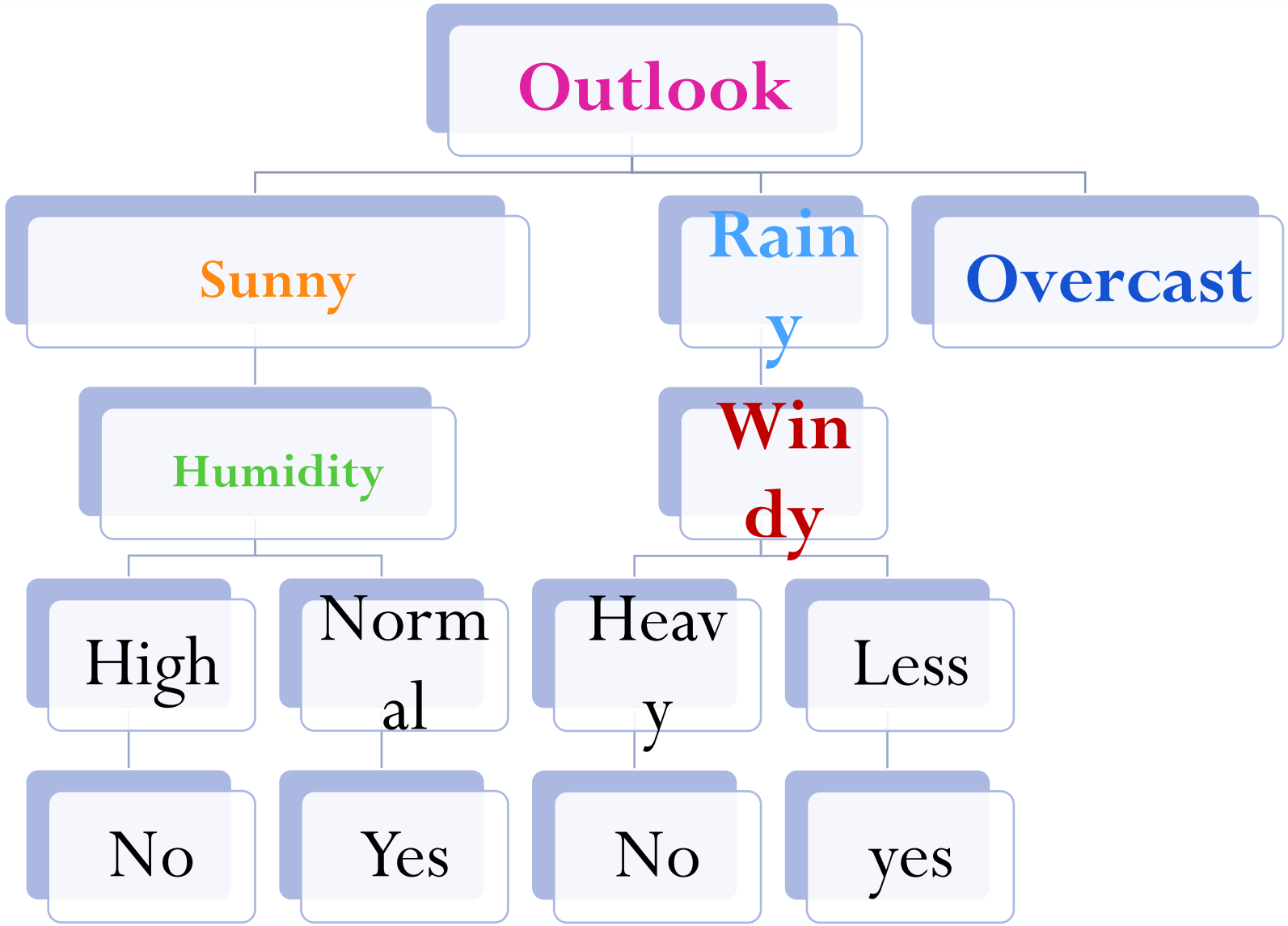
= 0.97

**Rainy Branch:**

Temperature, Windy

(Calculations show Windy has highest information gain)

# Decision Tree



## How to Make a Prediction?

Example: Outlook=Sunny, Temperature=Hot, Humidity=High, Windy=No

- Start at root: Outlook is Sunny
- Move to Humidity (High)
- Predict "No" (don't play cricket)

# Advantages of Decision Trees

- Easy to understand and interpret
- Can handle both numerical and categorical data
- Requires little data preprocessing
- Can model non-linear relationships

# Disadvantages

- Prone to overfitting
- Can be unstable with small variations in data
- May create biased trees if some classes dominate

# Decision Tree for Product Review Sentiment Classification

# SMARTPHONE PRODUCT REVIEWS

To demonstrate sentiment classification with a decision tree.

Example Dataset: Smartphone Reviews

Consider 12 real-world style smartphone reviews with these features:

- Contains "excellent": Binary feature (Yes/No)
- Contains "problem": Binary feature (Yes/No)
- Rating reference: Binary feature (Mentions rating ≥4: Yes/No)
- Sentiment: Target variable (Positive/Negative)

# Sample Dataset

| Review Text | Excellent | Problem | Rating≥4 | Sentiment |
|---|---|---|---|---|
| Excellent battery life | Yes | No | No | Positive |
| Camera has focusing problems | No | Yes | No | Negative |
| 5 stars - best phone I've owned | No | No | Yes | Positive |
| Screen stopped working after 2 days | No | Yes | No | Negative |
| Excellent performance, worth price | Yes | No | Yes | Positive |
| Charging port defective | No | Yes | No | Negative |

# Contd...

| Review Text | Excellent | Problem | Rating≥4 | Sentiment |
|---|---|---|---|---|
| 4.5/5 - excellent value for money | Yes | No | Yes | Positive |
| Software problems make it unusable | No | Yes | No | Negative |
| Excellent build quality | Yes | No | No | Positive |
| Would give 1 star if I could | No | No | No | Negative |
| 5/5 perfect in every way | No | No | Yes | Positive |
| Excellent but has minor heating issue | Yes | Yes | No | Positive* |

*Note: This is an interesting case where both positive and negative words appear

# Building the Decision Tree

Step 1: Calculate Parent Node Entropy

- Total instances: 12
- Positive: 7, Negative: 5

$$\text{Entropy} = -(7/12)*\log2(7/12) - (5/12)*\log2(5/12) \approx 0.98$$

# Contd...

**Step 2: Calculate Information Gain for Each Feature**

**1. Contains "excellent":**

- Yes: 5 (Positive:5, Negative:0)
- No: 7 (Positive:2, Negative:5)

Entropy(Yes) = 0 (pure node)

Entropy(No) = - (2/7)*log2(2/7) - (5/7)*log2(5/7) ≈ 0.86

**Information Gain** = 0.98 - [(5/12)*0 + (7/12)*0.86] ≈ 0.98 - 0.50 = 0.48

# Contd...

**2. Contains "problem":**

- Yes: 5 (Positive:1, Negative:4)
- No: 7 (Positive:6, Negative:1)

Entropy(Yes) = - (1/5)*log2(1/5) - (4/5)*log2(4/5) ≈ 0.72

Entropy(No) = - (6/7)*log2(6/7) - (1/7)*log2(1/7) ≈ 0.59

**Information Gain** = 0.98 - [(5/12)*0.72 + (7/12)*0.59] ≈ 0.98 - 0.64 = 0.34

# Contd...

**3. Mentions rating ≥4:**

- Yes: 4 (Positive:4, Negative:0)
- No: 8 (Positive:3, Negative:5)

Entropy(Yes) = 0 (pure node)

Entropy(No) = - (3/8)*log2(3/8) - (5/8)*log2(5/8) ≈ 0.95

**Information Gain** = 0.98 - [(4/12)*0 + (8/12)*0.95] ≈ 0.98 - 0.63 = 0.35

## Step 3: Select Root Node

- "Contains excellent" has the highest information gain (0.48), so it becomes our root node.

**Step 4: Build Sub-trees**

For "Contains excellent" = Yes branch (5 Positive, 0 Negative):

- Pure node (all Positive) - make this a leaf node

For "Contains excellent" = No branch (2 Positive, 5 Negative):

Current entropy = - (2/7)*log2(2/7) - (5/7)*log2(5/7) ≈ 0.86

# Contd...

**Calculate information gain for remaining features:**

**1. Contains "problem":**

- Yes: 4 (Positive:0, Negative:4)
- No: 3 (Positive:2, Negative:1)

Entropy(Yes) = 0

Entropy(No) = - (2/3)*log2(2/3) - (1/3)*log2(1/3) ≈ 0.92

Information Gain = 0.86 - [(4/7)*0 + (3/7)*0.92] ≈ 0.86 - 0.39 = 0.47

# Contd...

**2. Mentions rating ≥4:**

- Yes: 2 (Positive:2, Negative:0)
- No: 5 (Positive:0, Negative:5)

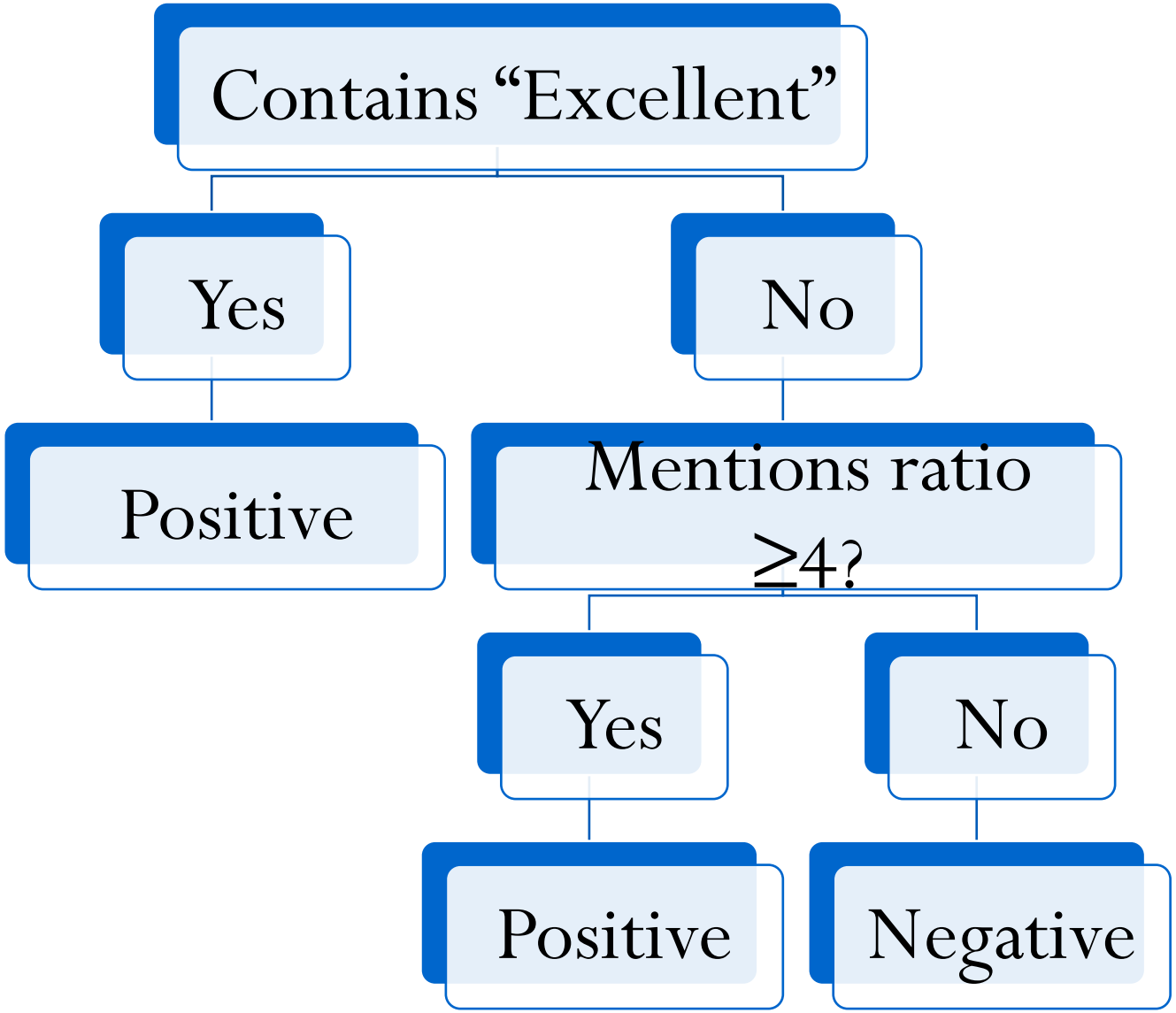
Entropy(Yes) = 0

Entropy(No) = 0

Information Gain = 0.86 - [(2/7)*0 + (5/7)*0] = 0.86 (highest)

- "Rating ≥4" has higher gain, so we select it.

# Final Decision Tree



# Making Predictions

**Example 1: "Excellent display quality"**

Contains "excellent"? Yes

→ Predict Positive

**Example 2: "My unit had battery problems"**

Contains "excellent"? No

Mentions rating ≥4? No

→ Predict Negative

**Example 3: "4 stars - decent phone"**

Contains "excellent"? No

Mentions rating ≥4?Yes

→ Predict Positive

**Example 4: "Not excellent but good enough"**

Contains "excellent"? No (notice negation isn't handled)

Mentions rating ≥4? No

→ Predict Negative (might be incorrect - shows limitation)

## Handling the Ambiguous Case

Our last training example was: "Excellent but has minor heating issue"

In our tree:

Contains "excellent"? Yes

→ Predict Positive (which matches our labeling)

# Contd...

- This shows that the "excellent" feature is so strong it overrides the negative aspect.

In practice, you might:

- Add more nuanced features ("but", "however" for contrast)
- Include sentiment intensity measures
- Use a more sophisticated model that can handle mixed sentiments